

Efficient Neural Networks on CIFAR10 dataset

Introduction

The purpose of this project is to create a neural network which can accurately predict the class of images from the CIFAR10 dataset. The neural networks that were trained in the various experiments were MLPs at first and CNNs combined with MLPs later on. The coding was done in Python using Numpy for math operations, PyPlot for display of training results and Torch for creating the CNN.

The dataset

The CIFAR-10 dataset consists of **60000 32x32 colour images in 10 classes**, with **6000 images** per class. There are **50000 training images** and **10000 test images**.

The dataset is divided into five training batches and one test batch, each with 10000 images. The test batch contains exactly 1000 randomly-selected images from each class. The training batches contain the remaining images in random order, but some training batches may contain more images from one class than another. Between them, the training batches contain exactly 5000 images from each class.

For example here is one image of the class “**airplane**” and one of the class “**cat**” (the images are 32x32 hence the low resolution).



Data Preprocessing

The training set was split into two sets, train (45000 images) and validation (5000 images). The data were also normalized using the mean and standard deviation on the dataset for each RGB channel as found on online documentation.

As experimental models started overfitting, the following augmentations were added to provide some variance to the data and avoid the problem:

- `transforms.RandomCrop(32, padding=4)`
- `transforms.RandomHorizontalFlip()`

- `transforms.ColorJitter(brightness=0.4,contrast=0.4,saturation=0.4)`
- `transforms.RandomRotation(15)`

Model Architecture

The final version of the trained CNN consists of 5 convolutional layers followed by two fully connected layers (hidden and output). For the convolutional layers the Kernel Size was 3x3 and the number of channels was gradually increasing as shown in the table below:

	Channels
1	3 → 32
2	32 → 64
3	64 → 128
4	128 → 256
5	256 → 256

After each layer Batch Normalization was applied so that the next layer receives a controlled distribution and Max Pooling was applied after layers 2, 4 and 5 to reduce high dimensionality leading from a 32x32 image to a 4x4 one. This way, the fully connected layer's input consists of $4 \times 4 \times 256 = 4096$ dimensions instead of $32 \times 32 \times 256 = 262,144$.

Following the convolution layers, there is one hidden layer of 1024 neurons ($4096 \rightarrow 1024$) and one output layer of 10 neurons, ($1024 \rightarrow 10$) as many as the classes that the dataset is split into. In these two layers a dropout rate of 0.5 was added in order to prevent overfitting. All layers except the output one use ReLU as activation. Below is code for the CNN's forward function:

```
def forward(self, x):
    x = F.relu(self.bn1(self.conv1(x)))
    x = self.pool(F.relu(self.bn2(self.conv2(x))))

    x = F.relu(self.bn3(self.conv3(x)))
    x = self.pool(F.relu(self.bn4(self.conv4(x))))

    x = self.pool(F.relu(self.bn5(self.conv5(x))))

    x = x.view(x.size(0), -1) # flatten

    x = self.dropout(x)
    x = F.relu(self.bn_fc1(self.fc1(x)))
    x = self.dropout(x)
    x = self.fc2(x)

    return x
```

Loss Function

For this project, Cross-Entropy Loss was chosen as the loss function because it is the standard and most effective choice for multi-class classification tasks such as CIFAR-10. This function is applied

to the unnormalized values coming from the output layer and calculates the negative log-likelihood of the input belonging to the corresponding class, after normalizing with Softmax:

$$\text{CrossEntropyLoss}(x, y) = -\ln\left(\frac{\exp(x_y)}{\sum \exp(x_j)}\right)$$

Optimizer

The model was trained using Stochastic Gradient Descent (SGD) with momentum, with a learning rate of 0.1, momentum of 0.9, and weight decay of 5×10^{-4} . In earlier versions of the model Adam optimizer was used. Although converging faster because it adapts an effective learning rate for each weight, this optimizer led to the model overfitting after 10-15 training epochs. On the other hand, SGD led to a slower training process (200 epochs for full training) while reaching a higher final validation accuracy.

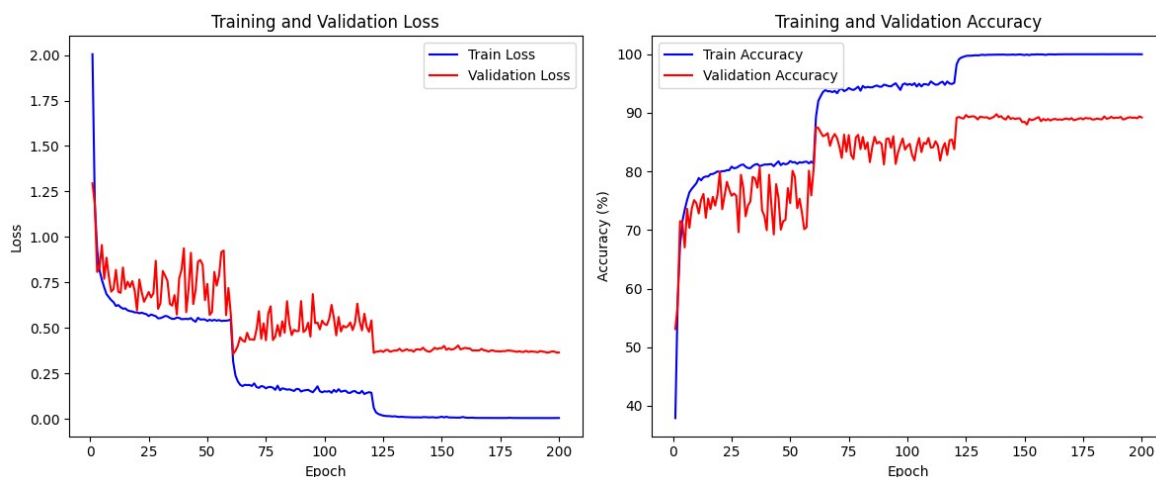
Additionally, a MultiStepLR scheduler was applied to reduce the learning rate at predetermined epochs (60, 120, 160), allowing the optimizer to make larger updates early on and finer adjustments as training progresses.

Results

After running 200 epochs for 52 minutes and 60 seconds on Google Collab using a T4 runtime, the results were as follows:

- Training Accuracy: 99.8%
- Training Loss: 0.0377
- Test Accuracy: 89.15% (8915/10000 predictions were correct)
- Test Loss: 0.3637

The following graphs were created with PyPlot to display the training process. Two major leaps seem to be happening in both loss and accuracy around epochs 60 and 120 respectively. These jumps align perfectly with the parameters of the Learning Rate scheduler and are caused by the increase in sensitivity as the learning rate drops. After epoch 130 there is no major further improvement on the validation loss function and some moderate overfitting is also observed.



Model Comparison

Below is a short table comparing the performance of 5 different models in terms of train and test accuracy and time efficiency. The first model was an MLP with two hidden layers of 512 and 256. It was later replaced by a CNN architecture as convolutional filters were proven to be more suitable for image datasets.

Model	Data Augmentation	Optimizer	Train Accuracy	Test Accuracy	Time
MLP 2 Hidden layers	No	Gradient Descent	59.59%	50.41%	265s
CNN 3 Conv layers	Yes	Adam	87.32%	78.60%	61s
CNN 3 Conv layers with Batch Normalization	Yes	Adam	91.71%	81.70%	95s
CNN 5 Conv layers with Batch Normalization	Yes	Adam	96.24%	86.49%	3m 42s
CNN 5 Conv layers with Batch Normalization (final model)	Yes	SGD	99.8%	89.15%	52m 60s

Nearest Neighbor – Nearest Centroid

Another comparison was conducted between this neural network and these three classification algorithms on the same dataset:

- 1. Nearest Neighbor (1-NN)
- 2. k-Nearest Neighbor (k-NN) with $k = 3$
- 3. Nearest Centroid Classifier

Model	Data Augmentation	Optimizer	Train Accuracy	Test Accuracy	Time
CNN 5 Conv layers with Batch Normalization (final model)	Yes	SGD	99.8%	89.15%	52m 60s
Nearest Neighbor (1-NN)	Normalization + PCA	Adam	96.24%	38.45%	122.75s
k-Nearest Neighbor (k-NN) with $k = 3$	Normalization + PCA	-	-	38.80%	123.60s
Nearest Centroid Classifier	Normalization + PCA	-	-	27.65%	0.12s

Conclusion

In this project, several neural network architectures were explored to classify images from the CIFAR-10 dataset, starting from simple MLPs and progressing toward deeper and more efficient CNN models. Through systematic experimentation involving data preprocessing, augmentation, architectural refinement, and optimizer selection, the final 5-layer CNN with batch normalization

and SGD achieved a strong test accuracy of **89.15%**, significantly outperforming classical methods such as k-Nearest Neighbors and Nearest Centroid. The results highlight the importance of convolutional feature extraction, regularization techniques, and careful tuning of training dynamics. Although some overfitting was observed in later stages of training, the model demonstrates robust generalization and showcases the effectiveness of deep convolutional architectures for image classification tasks.

References:

CIFAR-10 – University of Toronto

<https://www.cs.toronto.edu/~kriz/cifar.html>

PyTorch Documentation – Neural Network (torch.nn) Modules.

<https://pytorch.org/docs/stable/nn.html>

PyTorch Documentation – Torchvision Transforms.

<https://pytorch.org/docs/stable/torchvision/transforms.html>

PyTorch Documentation – Optimizers (torch.optim), including SGD and Adam.

<https://pytorch.org/docs/stable/optim.html>